

SecureShield

Mini Project II — Security Report

Name	Student ID
Barkhad Mohamed	210208908
Sabrin Ali Isack	210208994
Sadik Hassan Ismail	220208742

Course: Network Security

Project: Mini Project II — Role-Based Access Control API

Deadline: 03/05/2026

Question 1: Why is Salting Necessary to Prevent Rainbow Table Attacks?

What is a Rainbow Table Attack?

A Rainbow Table is a precomputed list of passwords and their corresponding hash values. Hackers use these tables to instantly look up what password matches a given hash. For example, if a hash always represents a known password, a hacker can crack it in seconds by looking it up in the table — without needing to guess anything.

The Problem Without Salting

Without salting, every user who has the same password will have the exact same hash stored in the database. This creates two serious problems:

- If a hacker gets the database, they can use a Rainbow Table to crack all passwords instantly.
- If two users share the same password, cracking one automatically cracks the other.

How Salting Solves This

A salt is a unique random value that is added to each password before it is hashed. This means even if two users have the exact same password, their stored hashes will be completely different because each has a different salt. See the example below:

User	Password	Salt	Result
Alice	pass123	x7k2mq	Unique Hash A
Bob	pass123	p9n4rs	Unique Hash B

This makes Rainbow Table attacks completely useless because the hacker would need a separate table for every possible salt value — which is computationally impossible. In our

SecureShield project, we used the bcrypt library which automatically generates a unique salt for every password and combines it with the hash before storing it in the database.

Question 2: Risks of Storing Sensitive Data Inside a JWT Payload

What is a JWT?

A JSON Web Token (JWT) consists of three parts separated by dots: Header, Payload, and Signature. Many developers mistakenly believe that JWT tokens are encrypted and therefore safe to store sensitive information inside. This is incorrect.

JWT is Encoded — NOT Encrypted

The JWT payload is only Base64 encoded — not encrypted. This means anyone who gets the token can instantly decode it and read all the data inside without needing any key or password. You can prove this yourself by pasting any JWT token into jwt.io and the payload will be immediately visible.

The Risks of Storing Sensitive Data in JWT:

- **Password Exposure:** If a password is stored inside the JWT, any attacker who intercepts the token can read it directly in plain text.
- **Personal Data Leaks:** Storing credit card numbers, emails, or personal identification numbers in a JWT is extremely dangerous because the data is readable by anyone who has the token.
- **Token Theft via XSS:** JWTs are often stored in browsers. If an attacker steals the token through a Cross-Site Scripting (XSS) attack, they gain access to all sensitive data stored inside it.
- **Long Token Lifetime:** JWTs are often valid for hours or days. If sensitive data is stored inside and the token is stolen, the attacker has prolonged access to that data until the token expires.

What We Did in SecureShield

In our project, we only stored the username and role inside the JWT — never the password or any other sensitive information. The JWT signature ensures the token was not tampered with, but it does NOT hide or encrypt the content. Sensitive data must always be kept on the server side and never placed inside a JWT payload.